# Présentation de l’application

ARGENTBANK

| Plan de la présentation | |
|---|---|
| Rappel du contexte | Page 2 |
| Structure du projet | Page 3 |
| Focus Redux | Page 4 |
| Swagger | Page 5 |
| Présentation code, visuel de l’application (mobile & desktop)+ devtools | Page 6 |
| Observations et Difficultés rencontrées | Page 7 |
| Bilan | Page 8 |

# Rappel du contexte

Bonjour,

En tant que nouveau développeur chez Remede Agency, vous nous avez confié une mission qui a pour objectif :

**Phase 1 : Création d'un nouveau système d'authentification des utilisateurs** dans application web permettant aux clients de « **Argent Bank** » de se connecter et de gérer leurs comptes et leur profil

Je dois principalement me centrer sur le développement front-end, l'intégration avec l'API et la gestion des states avec Redux.

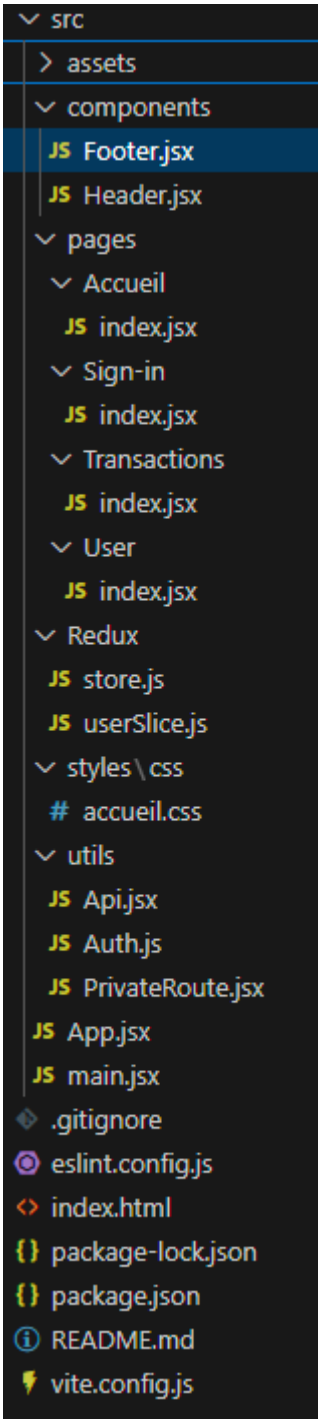
Je dispose pour cela d'un backEnd pour me connecter et de plusieurs issues pour suivre l'avancement.

Démarrer le projet en utilisant React pour une application responsive. Créer les pages Accueil, Sign-in et User selon les modèles HTML donnés dans le repo dont je dispose.

**Phase 2 : phase de conception de la page de Transactions**
Spécifier les endpoints d'API nécessaires pour une éventuelle seconde mission dans un fichier Swagger.

Objectif global : *Préparer une application opérationnelle et sécurisée qui servira de socle pour donner l'accès aux utilisateurs à la gestion de leurs comptes depuis l'application* « **ArgentBank** ».

# Structure du projet



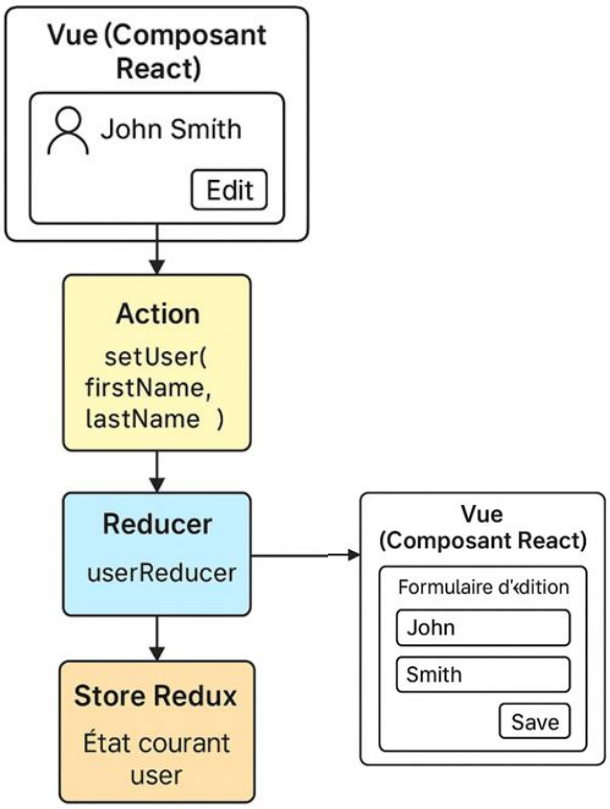

**Descriptions structure des fichiers**

- Les composants réutilisables dans src/components
- L'api.js centralise l'api via axios, Auth.js qui centralise les appels API et PrivateRoute.js pour la gestion privée de la route sont dans src/utils/
- Les codes Redux pour la définition du store et le Slice qui gère les données utilisateur dans src/Redux
- Les composants pages dans src/pages
- Les styles dans src/styles/css
- Images dans src/assets/img

# Redux

J'utilise Redux pour gérer **l'état utilisateur global** : nom, prénom, token. Cela me permet d'y accéder facilement depuis **plusieurs composants**, comme le header et la page profil, sans passer des props partout.

Schéma logique : Composant → dispatch(action) → reducer → store → re-render




| Etat du store | Mise à jour de la vue |
|---|---|
| « user » stocke prénoms, noms | Affiche les données dans la vue (header, page profil) |
| Action Redux dispatch(updateUser(updatedData)) est déclenché | le reducer de userSlice met à jour state.user.userInfo de façon immuable |
| | Le prénom et nom dans les inputs sont mis à jour |

"Lorsque l'utilisateur modifie son nom ou prénom, j'envoie une requête `PUT` via mon fichier `Auth.js`.
Une fois la réponse reçue, je **mets à jour le store Redux** en dispatchant l'action `updateUser` avec les nouvelles données.
Le reducer met alors à jour `state.user.userInfo`, ce qui déclenche automatiquement le re-render des composants connectés via `useSelector`, comme le **header** ou la **page profil**."

J'utilise `createSlice` de redux toolkit dans userSlice.js pour pour la gestion des utilisateurs pour les actions Redux

# Swagger

## User Module

| | | |
|---|---|---|
| POST | /user/login | Login |
| POST | /user/signup | Signup |
| POST | /user/profile | User Profile API |
| PUT | /user/profile | Update user profile |

## Account Module

| | | |
|---|---|---|
| GET | /user/accounts | Get user accounts |

## Transactions Module

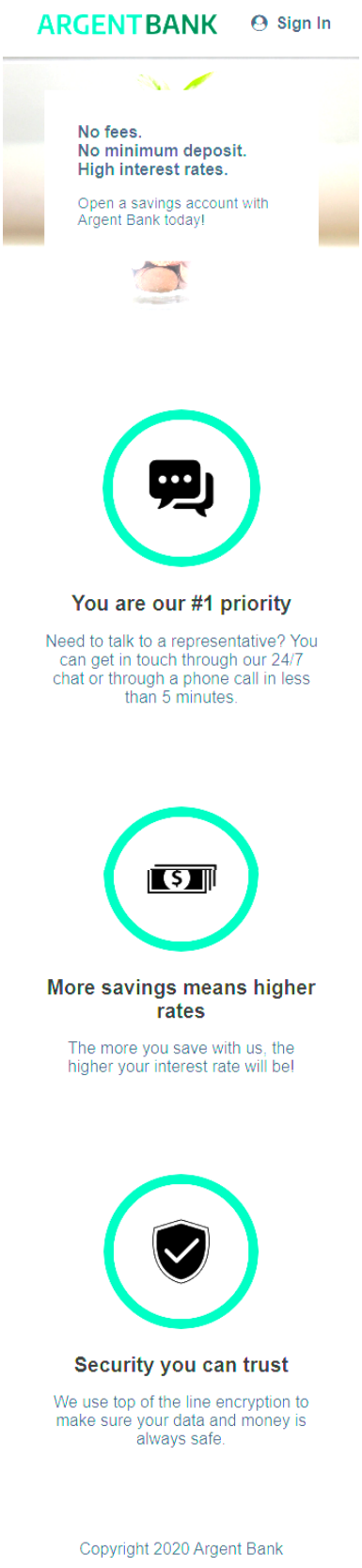
| | | |
|---|---|---|
| GET | /accounts/{accountId}/transactions | Get transactions for a specific account |
| GET | /accounts/{accountId}/transactions/{transactionId} | Get a specific transaction |
| PATCH | /accounts/{accountId}/transactions/{transactionId} | Update a transaction (transaction type, category or notes) |

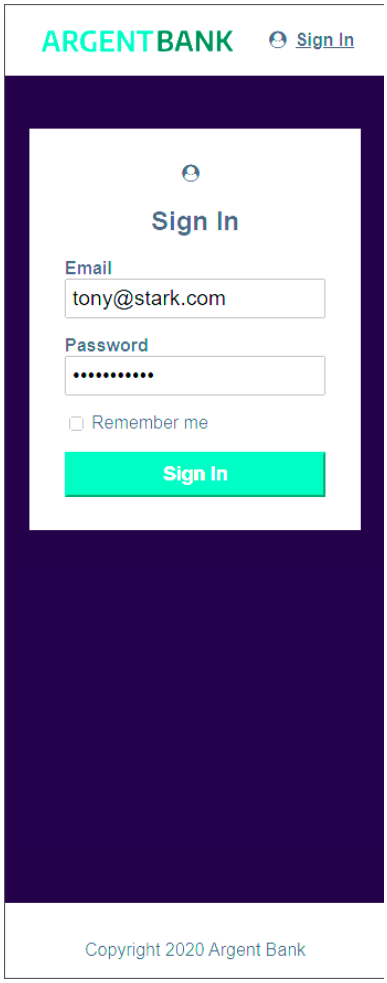
| | |
|---|---|
| POST /user/Login | • Utilisé pour le login |
| POST /user/signup | • Utilisé pour créer un nouvel utilisateur<br>• Le serveur genère un nouvel id |
| POST /user/profile | • utilisé pour la lecture du profil |
| PUT /user/profile | • modifier le prénom ou le nom via un formulaire<br>• Authentification requise (JWT Token) |
| GET /accounts/{accountId} /transactions | • Retourne la liste des transactions d'un compte<br>• Supporte les filtres optionnels via query string (1*) |
| GET /accounts/{accountId} /transactions/ {transactionId} | • Retourne le détail des transactions d'un compte |
| PATCH /accounts/{accountId} /transactions/ {transactionId} | • modification partielle : category, notes, éventuellement transaction type<br>• Pas de suppression mais envoi de null pour vider les champs (2*) |

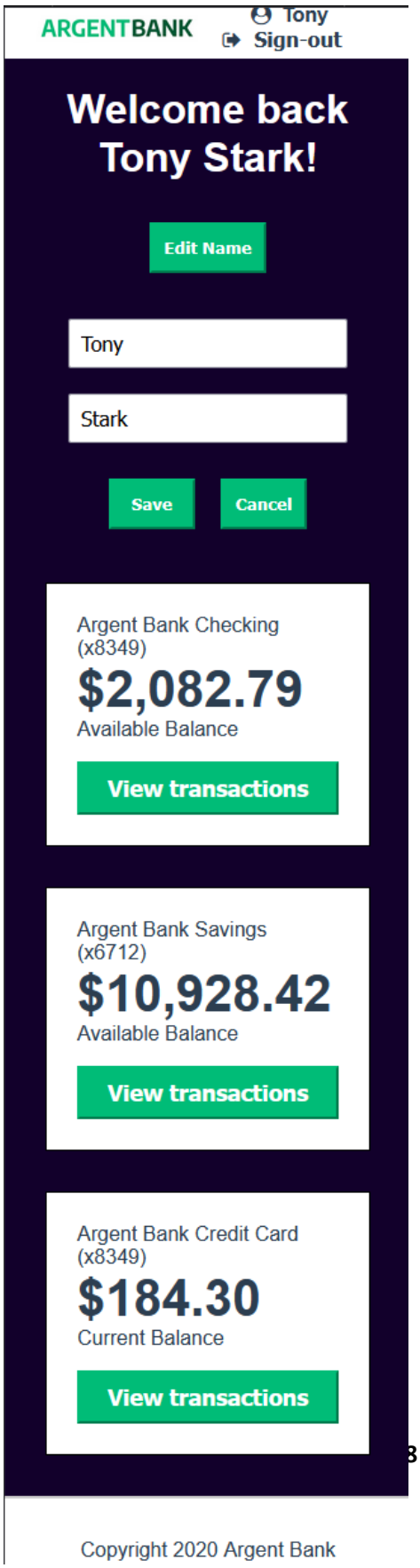
1* : http: /accounts/{accountId}/transactions?startDate=2025-01-01&endDate=2025-01-31 ou en logique backend: if (startDate && endDate) {// filtre entre deux dates} else {// retourner toutes les transactions}

2* :Logique backend: { "transactionType": null,"category": null, "notes": null}

# Présentation du code et du visuel de l'application (mobile & desktop) et de devtools Redux



# Observations

**Axios :**
J'ai préféré Axios à fetch car sa syntaxe est plus claire, et il gère automatiquement des aspects importants comme :

- la conversion des réponses JSON,
- la gestion des erreurs HTTP (404, 500, etc.).

Ce choix rend le code plus lisible et plus maintenable dans un projet destiné à évoluer.

**Redux :**
Redux Toolkit propose createAsyncThunk pour gérer les appels asynchrones plus proprement.
Ce n'était pas requis ici, mais j'aurais pu :

- gérer les états pending, fulfilled, rejected d'une requête API,
- simplifier le code lié à la connexion ou à la récupération du profil utilisateur,
- afficher automatiquement des messages d'erreur.

Cela aurait rendu l'architecture plus robuste.

**Swagger :**
La méthode PUT /user/profile devrait plutôt être une PATCH, puisque la ressource n'est pas toujours mise à jour en totalité.

Le champ transaction type devrait selon la logique bancaire être modifiable par l'utilisateur.

**Commentaires** :
J'ai commenté le code en français pour cette présentation, mais il peut être traduit en anglais si besoin.

## Difficultés rencontrées

- **Compatibilité Node.js** : le Backend exige Node 12, alors que le frontend nécessite Node 16+.
  J'ai donc utilisé une version Dockerisée du backend pour éviter les conflits.

- **Swagger** : sa syntaxe étant très sensible à l'indentation, je me suis documenté pour garantir la lisibilité du fichier.

# Bilan

Ce troisième projet React m’a permis de :

- renforcer mes compétences en appels API,
- apprendre à structurer l’état global avec Redux et Redux Toolkit.
- **Redux** permet de centraliser les données utilisateur (nom, prénom, token).

  Les composants accèdent facilement à ces données via useSelector.

  Les **Redux DevTools** aident à visualiser les actions et à suivre l’évolution de l’état en temps réel.

Enfin, Swagger m’a permis de mieux comprendre la structure des API REST et la logique des requêtes/ réponses.

L'application est stable et **prête pour les évolutions futures demandées**.